

بسمی تعالی



دانشگاه شهید باهنر کرمان

پروپوزال پروژه درس نظریه زبان ها و ماشین ها

(تشخیص گرامر منظم و تبدیل آن به DFA)

نام استاد: خانم دکتر شیما شفیعی

تاریخ: ۱۴۰۵/۰۳/۱۷

نیمسال تحصیلی اول ۱۴۰۴-۱۴۰۵

عنوان پروژه:

طراحی و پیاده سازی سیستمی برای تشخیص منظم یا نامنظم بودن گرامر و رسم DFA متناظر

اعضای تیم:

(گروه شماره ۳)

حامد یزدی زاده:

- بخش منطق (بک اند): پیاده سازی ساختار پایه و تعریف اولیه کلاس ها
- بخش ویندوز فرم: پیاده سازی ابتدایی کلاس MainForm و تکمیل متد initializeComponent
- شماره دانشجویی: ۴۰۳۴۰۵۰۷۵

علی ادیب رضوی:

- بخش منطق (بک اند): تکمیل متد های درون کلاس ها
- بخش ویندوز فرم: تکمیل متد PnlGraph_paint و BtnAnalyze_click
- شماره دانشجویی: ۴۰۳۴۰۵۰۰۲

سجاد صباغ کرمانی:

- بخش منطق (بک اند): تکمیل متد های درون کلاس ها
- بخش ویندوز فرم: تکمیل متد BtnAnalyze و DrawArrow
- شماره دانشجویی: ۴۰۳۴۰۵۰۴۶

بیان مسئله:

نظریه زبان ها و ماشین ها یکی از پایه های اصلی علوم کامپیوتر است که به مطالعه زبان ها و مدل های محاسباتی مربوط به آن ها می پردازد. گرامرها ابزاری قدرتمند برای توصیف ساختار زبان ها هستند و بر اساس پیچیدگی و ساختار قواعد تولید، به دسته های مختلفی تقسیم می شوند. گرامرهای منظم، که ساده ترین دسته از گرامرهای مستقل از متن هستند، با زبان های منظم و ماشین های حالت متناهی قطعی (DFA) ارتباط تنگاتنگی دارند.

با این حال، تشخیص اینکه یک گرامر داده شده، منظم است یا خیر، و در صورت منظم بودن، تبدیل آن به یک DFA متناظر، می‌تواند یک فرآیند دستی زمان‌بر و مستعد خطا باشد، به خصوص برای گرامرهای بزرگ. این امر، نیاز به توسعه ابزارهایی را که بتوانند این فرآیند را به صورت خودکار انجام دهند، برجسته می‌سازد.

این پروژه با هدف طراحی و پیاده‌سازی یک سیستم نرم‌افزاری انجام می‌شود که قادر است یک گرامر را به عنوان ورودی دریافت کرده، ابتدا منظم یا نامنظم بودن آن را تشخیص دهد و در صورتی که گرامر منظم تشخیص داده شد، DFA معادل آن را تولید و به صورت گرافیکی نمایش دهد. این سامانه می‌تواند به عنوان یک ابزار آموزشی مؤثر و کاربردی برای دانشجویان استفاده قرار گیرد.

اهداف پروژه	ضرورت و اهمیت پروژه
دریافت گرامر از کاربر به صورت متنی	کمک آموزشی به دانشجویان برای درک بهتر گرامرهای منظم و DFA
بررسی قواعد گرامر و تشخیص منظم یا نامنظم بودن آن	کاهش خطاهای انسانی در تشخیص نوع گرامر
در صورت منظم بودن گرامر، استخراج حالت‌ها و انتقال‌ها	افزایش سرعت تحلیل گرامرهای ورودی
ساخت DFA متناظر با گرامر	ایجاد ارتباط عملی بین مباحث نظری و پیاده‌سازی نرم‌افزاری
رسم گرافیکی DFA و نمایش نتیجه به کاربر	امکان مشاهده بصری DFA برای فهم بهتر ساختار زبان

توضیح الگوریتم تبدیل گرامر منظم به DFA :

- تعریف DFA: یک مدل محاسباتی است که از مجموعه‌ای از حالات، یک الفبای ورودی، یک تابع انتقال، یک حالت اولیه و مجموعه‌ای از حالات پذیرش تشکیل شده است. DFA رشته‌ها را با دنبال کردن مسیرهای انتقال بین حالات بر اساس نمادهای ورودی پردازش می‌کند و اگر در پایان پردازش یک رشته، در یکی از حالات پذیرش باشد، آن رشته را می‌پذیرد.

برای تبدیل یک گرامر منظم به یک ماشین حالت متناهی قطعی، مراحل زیر باید دنبال شوند:

اگر گرامر منظم ما $G = (V, T, P, S)$ باشد که در آن:

- V : مجموعه متغیرها (non-terminals)
- T : مجموعه نمادهای ترمینال (terminals)
- P : مجموعه قواعد تولید (production rules)

• S : متغیر شروع (start variable)

برای تبدیل این گرامر به یک DFA $M = (Q, \Sigma, \delta, q_0, F)$ مراحل زیر را انجام می‌دهیم:

۱. تعیین مجموعه‌ی حالات (Q):

- برای هر متغیر $A \in V$ در گرامر، یک حالت (state) در DFA خواهیم داشت.
- یک حالت اضافی، حالت شروع DFA (که معمولاً با S نشان داده می‌شود)، را نیز اضافه می‌کنیم. این حالت اضافی، شروع پردازش را نشان می‌دهد و با متغیر شروع گرامر (S) مرتبط است.
- اگر گرامر قاعده تولید $A \rightarrow \epsilon$ داشته باشد (و A تنها متغیر سمت چپ آن باشد)، حالت متناظر با A (q_A) یکی از حالات پذیرش خواهد بود.
- اگر متغیر شروع گرامر (S) بتواند مستقیماً به رشته تهی (ϵ) تبدیل شود (یعنی قاعده $S \rightarrow \epsilon$ وجود داشته باشد)، حالت اولیه DFA (S) یک حالت پذیرش خواهد بود.

۲. تعیین الفبای ورودی (Σ):

- الفبای ورودی DFA دقیقاً همان مجموعه نمادهای ترمینال (Σ) گرامر است.

۳. تعیین حالت اولیه (S):

- حالت اولیه DFA، همان حالت متناظر با متغیر شروع گرامر، یعنی S خواهد بود.

۴. تعیین مجموعه‌ی حالات پذیرش (F):

- برای هر متغیر $A \in V$ که قاعده $A \rightarrow \epsilon$ در گرامر وجود داشته باشد (و A تنها متغیر در سمت چپ آن باشد)، حالت q_A را به مجموعه حالات پذیرش F اضافه می‌کنیم.
- اگر قاعده $S \rightarrow \epsilon$ وجود داشته باشد، حالت اولیه q_S (که همان S است) را نیز به F اضافه کنید.

۵. تعیین تابع انتقال (δ):

- تابع انتقال δ را بر اساس قواعد تولید گرامر به صورت زیر تعریف می‌کنیم:
- برای قاعده $A \rightarrow aB$: تابعی تعریف می‌کنیم که از حالت A با دریافت نماد ورودی a به حالت B برود. یعنی: $\delta(A, a) = B$.
- برای قاعده $A \rightarrow a$: تابعی تعریف می‌کنیم که از حالت A با دریافت نماد ورودی a به یک حالت پذیرش نهایی برود. این حالت پذیرش نهایی، در واقع همان حالتی است که متناظر با پذیرش رشته در انتهای یک مسیر معتبر است. در برخی روش‌های تبدیل، یک حالت پذیرش جدید و مشترک (مثلاً q_{accept}) ایجاد می‌شود و در اینجا $\delta(q_A, a) = q_{accept}$. اگر حالت q_A نیز در F باشد، آنگاه q_{accept} نیز باید در F قرار گیرد. در روش متداول‌تر، اگر $A \rightarrow a$ تنها راه

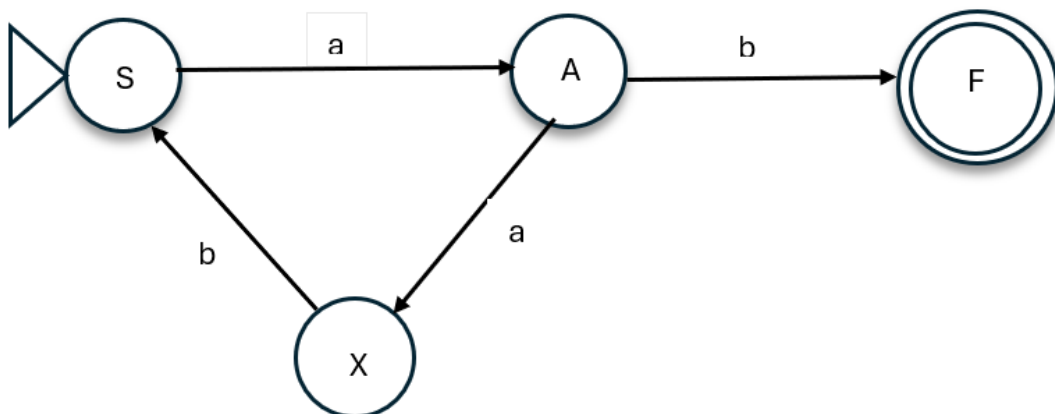
رسیدن به پایان رشته از A باشد، A خود در F قرار می‌گیرد و اما ساده‌ترین راه این است که اگر $A \rightarrow a$ داریم، حالت A را در F قرار دهیم و برای a به یک حالت نهایی برویم. بطور خلاصه روش این است که ابتدا تمام حالات متناظر با متغیرها را در نظر بگیریم، سپس حالات پذیرش را بر اساس قواعد ϵ تعیین کنیم، و در نهایت انتقال‌ها را تعریف کنیم.

چند مثال برای درک بیشتر توضیحات:

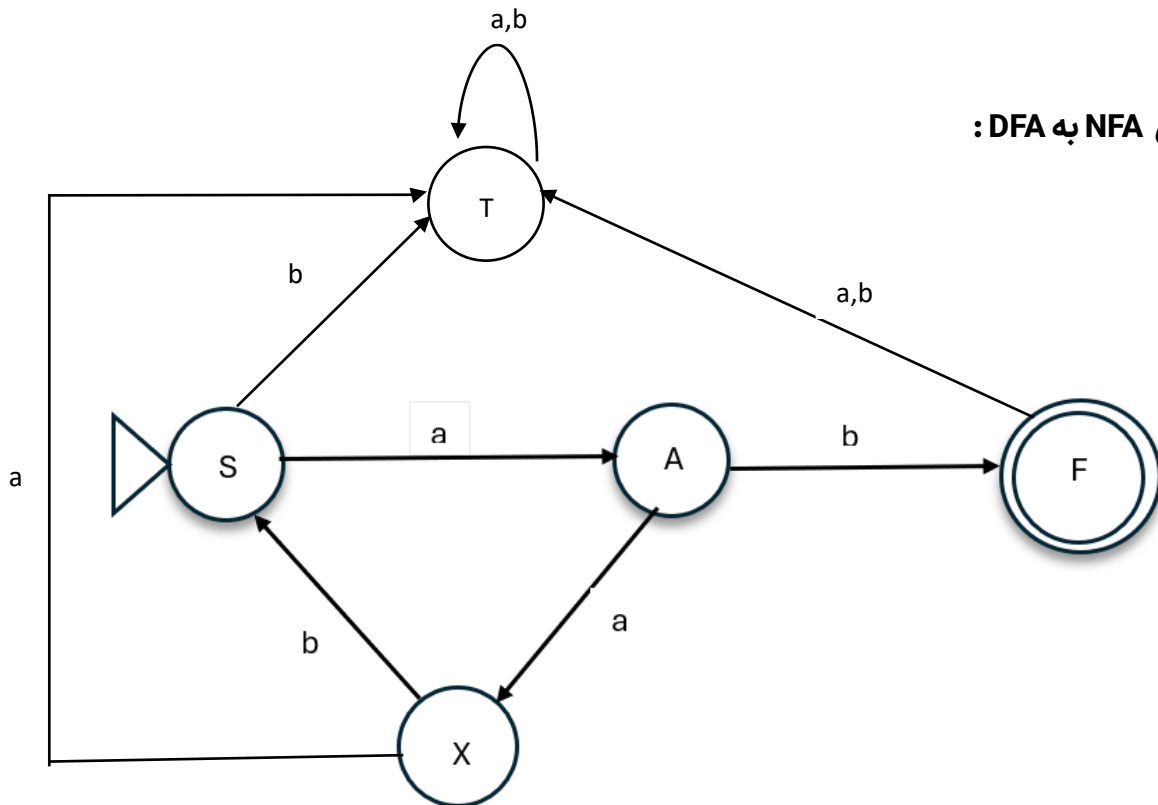
مثال ۱:

$S \rightarrow aA$	$\Rightarrow$	$S \rightarrow aA$
$A \rightarrow abS \mid b$		$A \rightarrow aX \mid b$
		$X \rightarrow bS$

گراف NFA مثال:



تبدیل DFA به NFA :

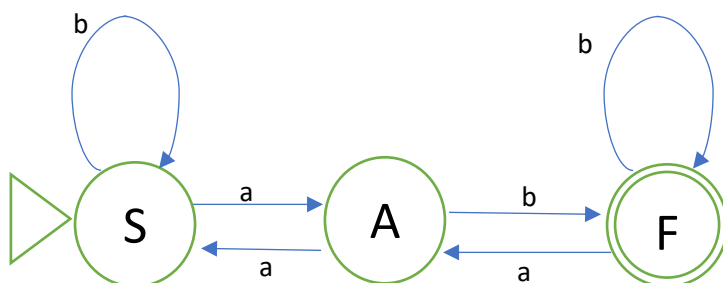


مثال ۲:

$S \rightarrow aA \mid bS$

$A \rightarrow aS \mid bF$

$F \rightarrow aA \mid bF \mid \lambda$



نحوه عملکرد پروژه:

در این پروژه، کاربر گرامر مورد نظر را وارد می کند. سیستم ابتدا ساختار قواعد گرامر را بررسی می کند و مشخص می سازد که آیا گرامر از نوع منظم است یا خیر. برای این کار، قواعد باید با الگوهای گرامر منظم مطابقت داشته باشند.

اگر گرامر نامنظم باشد، سیستم این موضوع را اعلام می کند و از ادامه ی تبدیل جلوگیری می کند.

اگر گرامر منظم باشد، سیستم با استخراج حالت‌های لازم، انتقال‌ها را شناسایی کرده و DFA متناظر را تولید می‌کند. سپس این DFA به صورت گرافیکی رسم و نمایش داده می‌شود.

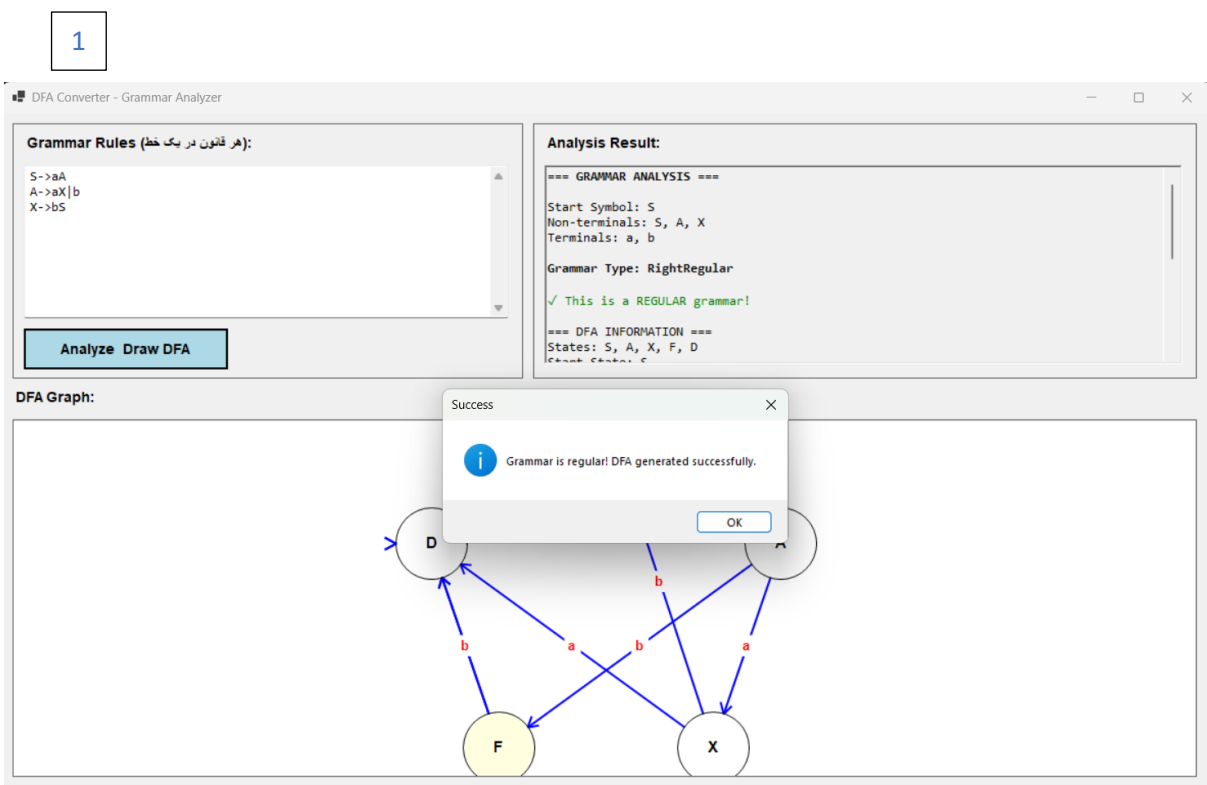
ترتیب انجام کار به صورت پایین است:

- کاربر گرامر را در قالب مشخص وارد می‌کند.
- برنامه قواعد را بررسی می‌کند تا مشخص شود هر تولید با ساختار گرامر منظم سازگار است یا نه.
- اگر همه‌ی تولیدها مطابق قواعد گرامر منظم باشند، گرامر منظم تشخیص داده می‌شود؛ در غیر این صورت، نامنظم است.
- در صورت منظم بودن گرامر، حالت‌ها و انتقال‌ها از روی تولیدها استخراج می‌شوند و DFA ساخته می‌شود.
- DFA نهایی به کمک ابزارهای مناسب به صورت بصری نمایش داده می‌شود.

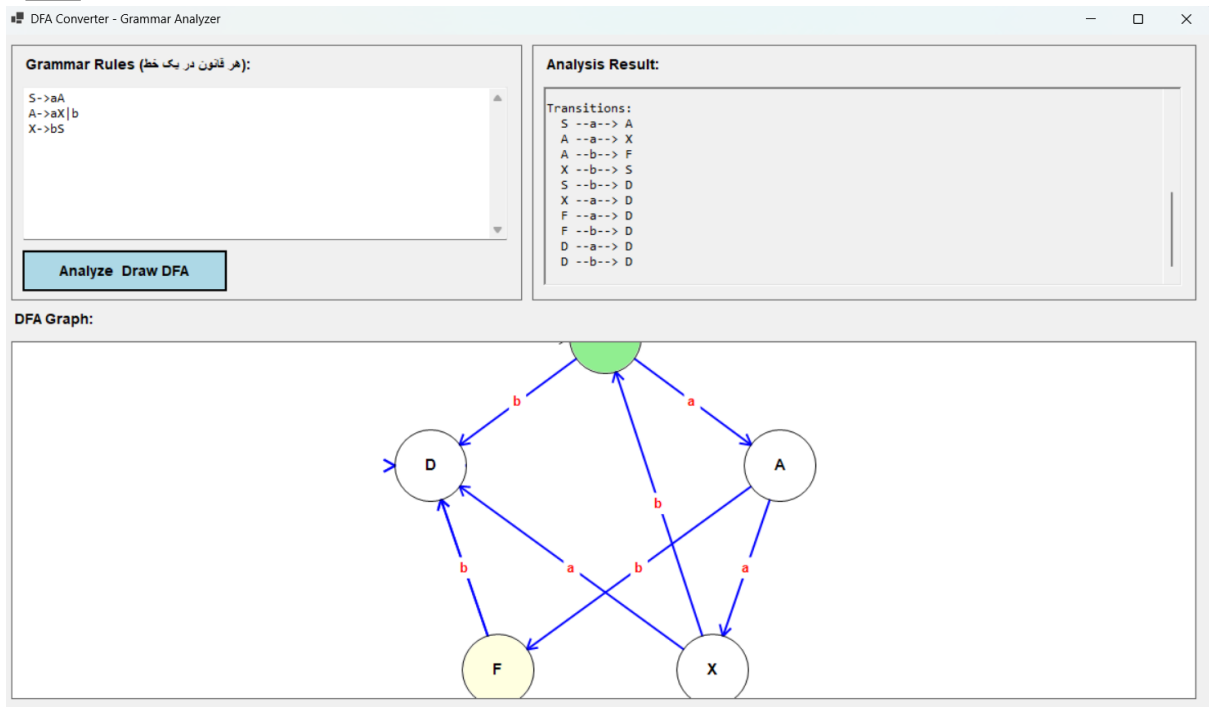
اجرای برنامه و دریافت خروجی:

گرامر به فرم مشخص در کادر مربوطه نوشته می‌شود و سپس بر روی دکمه Analyze Draw DFA زده می‌شود در کادر پایین صفحه اگر گرامر منظم باشد گراف کشیده می‌شود ولی اگر نامنظم باشد پیغام ارور داده می‌شود و گرافی کشیده نمی‌شود. در کادر سمت راست صفحه توضیحات جزئی در مورد گرامر و رسم گراف داده می‌شود.

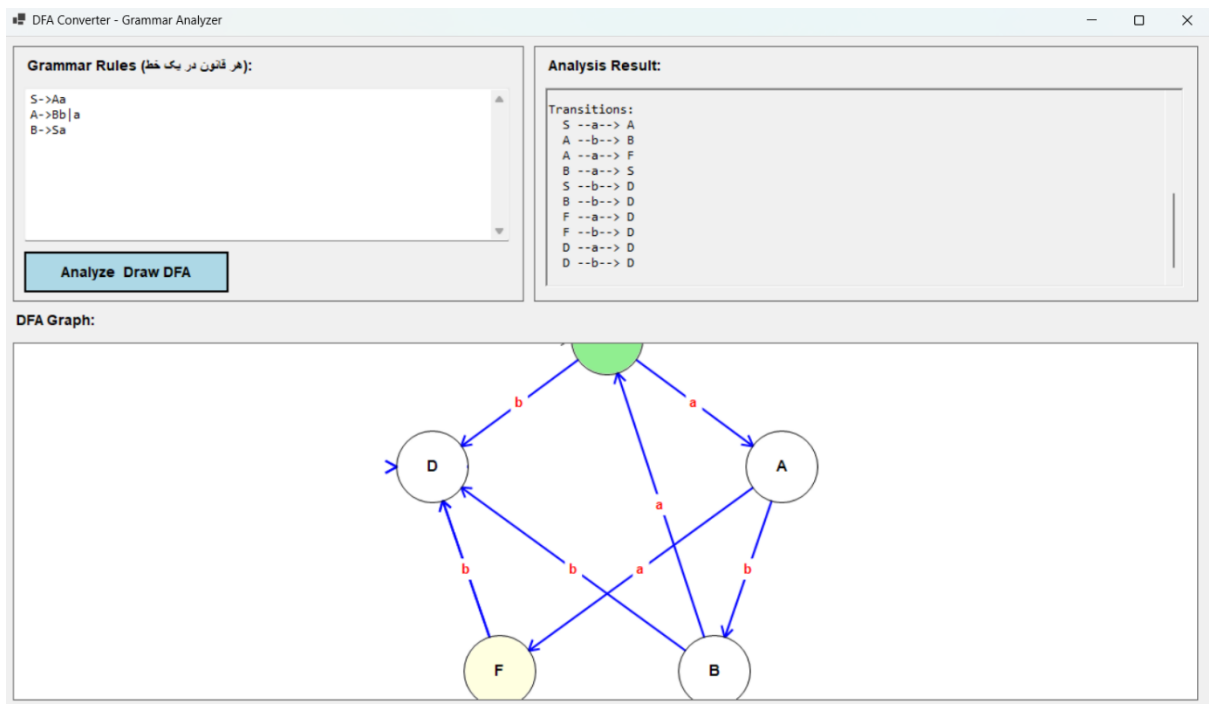
چند نمونه ورودی و خروجی به این شکل می‌باشد:

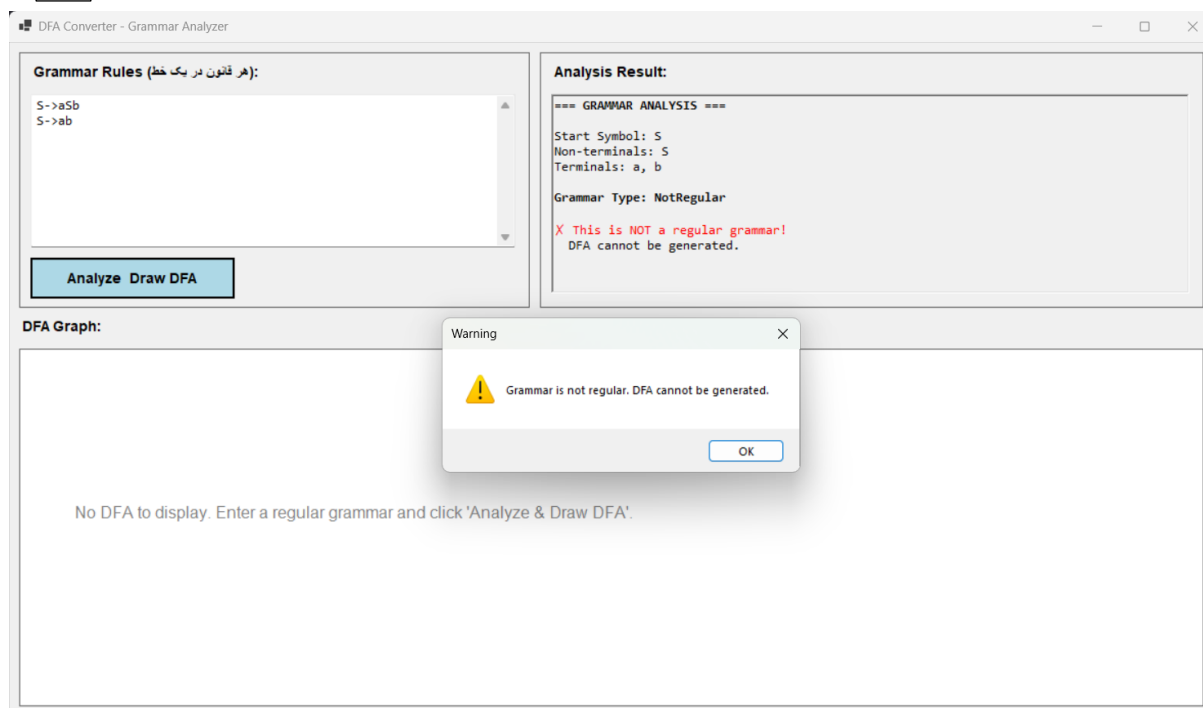


1



2





توضیحات مربوط به ورودی و خروجی ها:

- در تصویر شماره ۱ گرامر راست خط است که منظم تشخیص داده شده و گراف رسم شده است.
- در تصویر شماره ۲ گرامر چپ خط (منظم) است و گراف آن رسم شده است.
- در تصویر شماره ۳ گرامر نامنظم است و پیغام هشدار داده شده است و گرافی نمیتواند برای آن رسم کند.

ابزار های مورد استفاده:

- استفاده از زبان برنامه نویسی C# برای پیاده سازی منطق
- استفاده از محیط برنامه نویسی VS Code
- استفاده از Window Forms برای طراحی بصری گراف DFA

نتیجه گیری:

پروژه‌ی حاضر با هدف بررسی و تحلیل گرامرهای ورودی و تبدیل گرامرهای منظم به DFA طراحی شد. این سیستم علاوه بر کاربرد آموزشی، می‌تواند به عنوان ابزاری برای بررسی خودکار گرامرها و نمایش تصویری ماشین متناظر مورد استفاده قرار گیرد. اجرای این پروژه موجب درک بهتر مفاهیم نظری و کاربردی زبان‌های رسمی خواهد شد.

منابع و لینک کد ها:

<https://gitlab.aranserver.com/SajadSK/project-nzm>

کتاب: An Introduction to Formal Languages and Automata – 7th Edition (Peter Linz)